Practical – 11

Aim-1: Save/load

Make a small class Serializable, save an array of its objects to a file with ObjectOutputStream, then read it back into fresh objects and confirm the values survived. Mark one field transient and show it is not saved.

Code :

Main.java

```java
import java.io.*;

class Student implements Serializable {

    private int id;
    private String name;

    // This field will NOT be saved
    private transient double marks;

    Student(int id, String name, double marks) {
        this.id = id;
        this.name = name;
        this.marks = marks;
    }

    public void display() {
        System.out.println("ID     : " + id);
        System.out.println("Name   : " + name);
        System.out.println("Marks  : " + marks);
    }
}

public class Main {

    public static void main(String[] args) {

        String fileName = "students.dat";

        try {
```

Terminal

Main.java

```
public class Main {
    public static void main(String[] args) {
        try {
            // Create objects
            Student[] students = {
                new Student(101, "Kavya", 85.5),
                new Student(102, "Rahul", 90.0),
                new Student(103, "Priya", 78.5)
            };

            // Save objects to file
            ObjectOutputStream out =
                new ObjectOutputStream(new FileOutputStream(fileName));

            out.writeObject(students);
            out.close();

            System.out.println("Objects saved successfully.\n");

            // Read objects from file
            ObjectInputStream in =
                new ObjectInputStream(new FileInputStream(fileName));

            Student[] loadedStudents =
                (Student[]) in.readObject();

            in.close();

            System.out.println("Objects loaded successfully:");

            for (Student s : loadedStudents) {
```

Terminal

Main.java

```
public class Main {
    public static void main(String[] args) {
        try {
            System.out.println("Objects saved successfully.\n");

            // Read objects from file
            ObjectInputStream in =
                new ObjectInputStream(new FileInputStream(fileName));

            Student[] loadedStudents =
                (Student[]) in.readObject();

            in.close();

            System.out.println("Objects loaded successfully:");

            for (Student s : loadedStudents) {
                s.display();
                System.out.println();
            }

            System.out.println(
                "Notice: Marks became 0.0 because it was transient."
            );

        } catch (IOException | ClassNotFoundException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Output :

```
Objects saved successfully.

Objects loaded successfully:
ID      : 101
Name    : Kavya
Marks   : 0.0

ID      : 102
Name    : Rahul
Marks   : 0.0

ID      : 103
Name    : Priya
Marks   : 0.0

Notice: Marks became 0.0 because it was transient.
```

Learning Outcomes :

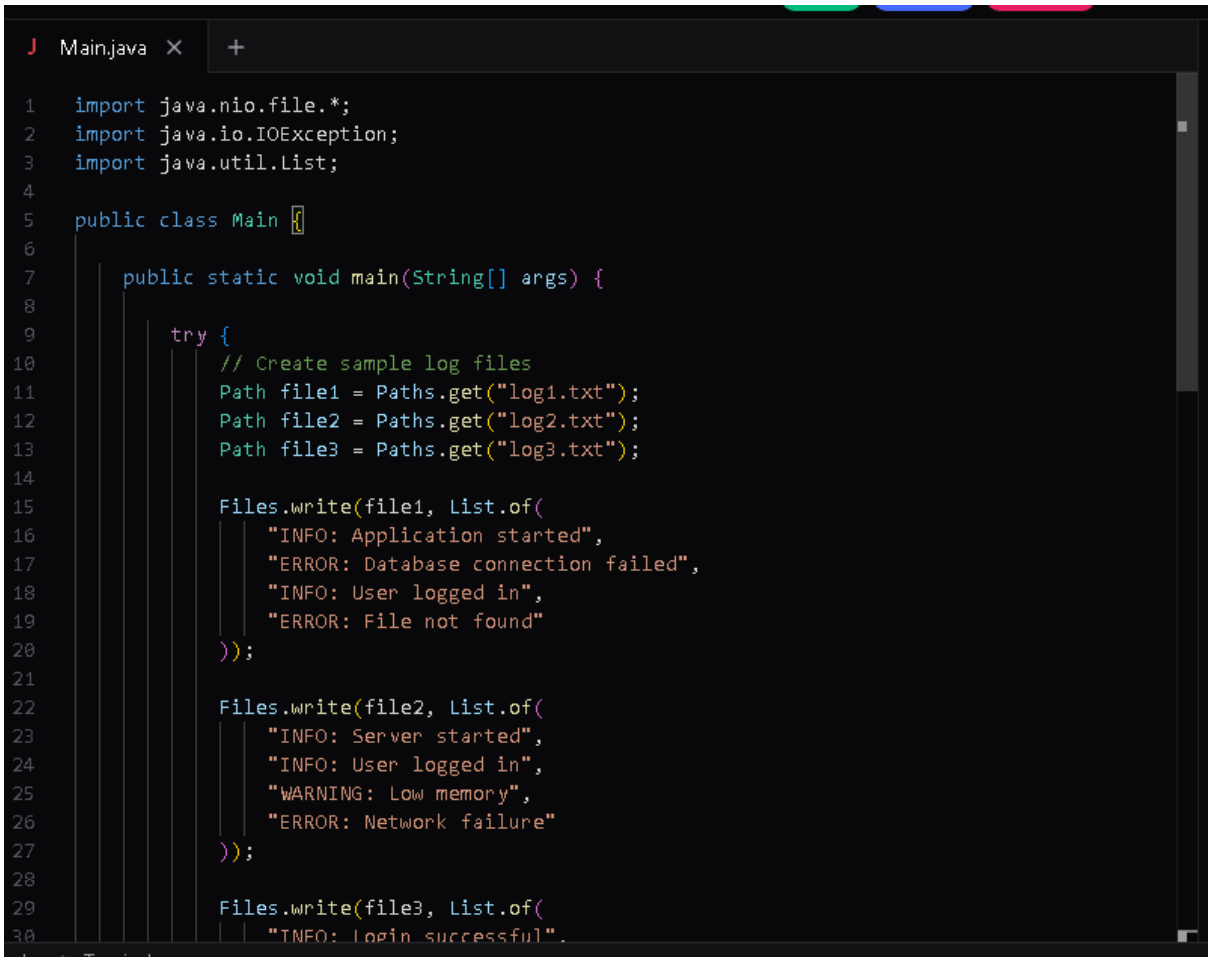
After completing this program, students will be able to:

1. Understand **serialization and deserialization** in Java.
2. Use `Serializable` to make objects savable.
3. Use `ObjectOutputStream` to write objects to a file.
4. Use `ObjectInputStream` to read objects back from a file.
5. Understand the purpose of the **`transient` keyword**.
6. Verify that serialized object values are preserved after loading.

Aim- 2 : Log analyzer

Create a few text files, then use Path/Files to read each line by line and count the total lines and how many contain a keyword across all files; also print each file's size from its attributes.

Code :



```java
import java.nio.file.*;
import java.io.IOException;
import java.util.List;

public class Main {

    public static void main(String[] args) {

        try {
            // Create sample log files
            Path file1 = Paths.get("log1.txt");
            Path file2 = Paths.get("log2.txt");
            Path file3 = Paths.get("log3.txt");

            Files.write(file1, List.of(
                "INFO: Application started",
                "ERROR: Database connection failed",
                "INFO: User logged in",
                "ERROR: File not found"
            ));

            Files.write(file2, List.of(
                "INFO: Server started",
                "INFO: User logged in",
                "WARNING: Low memory",
                "ERROR: Network failure"
            ));

            Files.write(file3, List.of(
                "INFO: Login successful",
```

```java
public class Main {
    public static void main(String[] args) {
        try {
            Files.write(file3, List.of(
                "INFO: Login successful",
                "ERROR: Invalid password",
                "INFO: Data saved"
            ));

            Path[] files = {file1, file2, file3};

            String keyword = "ERROR";

            int totalLines = 0;
            int keywordLines = 0;

            System.out.println("LOG ANALYSIS");
            System.out.println("------------");

            for (Path file : files) {

                List<String> lines = Files.readAllLines(file);

                int fileKeywordCount = 0;

                for (String line : lines) {

                    totalLines++;

                    if (line.contains(keyword)) {
```

Terminal

Main.java

```
public class Main {
    public static void main(String[] args) {
        try {
            for (Path file : files) {
                for (String line : lines) {
                    }
                }

                long size = Files.size(file);

                System.out.println("File: " + file);
                System.out.println("Size: " + size + " bytes");
                System.out.println(
                    "Lines containing " + keyword + ": "
                    + fileKeywordCount
                );
                System.out.println();
            }

            System.out.println("Total lines: " + totalLines);
            System.out.println(
                "Total lines containing " + keyword + ": "
                + keywordLines
            );

        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Terminal

Output :

```
LOG ANALYSIS
------------
File: log1.txt
Size: 103 bytes
Lines containing ERROR: 2

File: log2.txt
Size: 85 bytes
Lines containing ERROR: 1

File: log3.txt
Size: 64 bytes
Lines containing ERROR: 1

Total lines: 11
Total lines containing ERROR: 4
```

Learning Outcomes

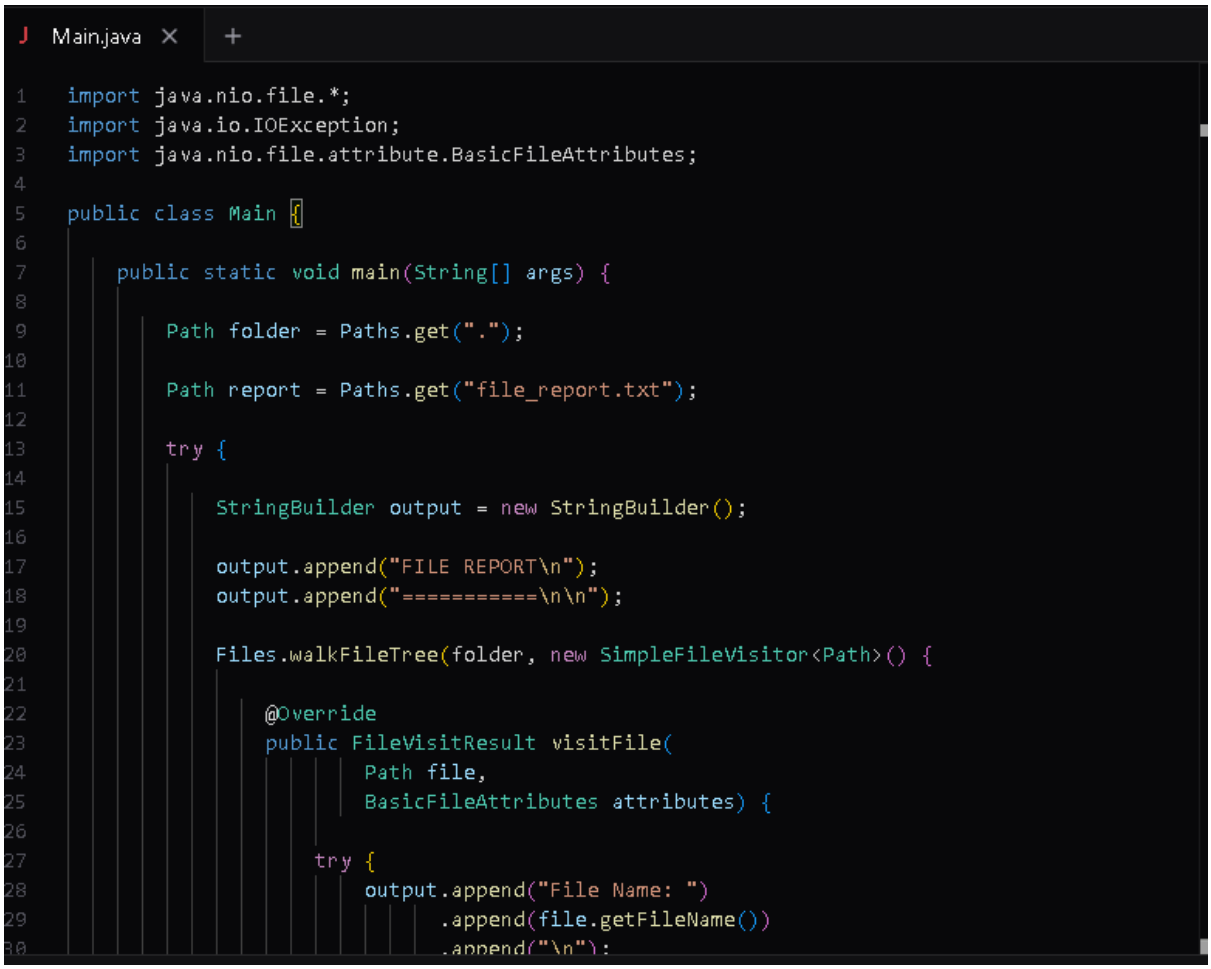
After completing this program, students will be able to:

1. Use Java NIO's **`Path` and `Files`** classes.
2. Create and read text files using Java.
3. Process files **line by line**.
4. Count total lines across multiple files.
5. Search for a specific **keyword** in text files.
6. Retrieve file information such as **file size**.

Aim – 3 : For Advanced Learners – home

Walk a folder tree with NIO and produce a report of the files it contains (name, size, modified time), writing the summary to a file; decide what to include.

Code :



```java
import java.nio.file.*;
import java.io.IOException;
import java.nio.file.attribute.BasicFileAttributes;

public class Main {

    public static void main(String[] args) {

        Path folder = Paths.get(".");

        Path report = Paths.get("file_report.txt");

        try {

            StringBuilder output = new StringBuilder();

            output.append("FILE REPORT\n");
            output.append("===========\n\n");

            Files.walkFileTree(folder, new SimpleFileVisitor<Path>() {

                @Override
                public FileVisitResult visitFile(
                        Path file,
                        BasicFileAttributes attributes) {

                    try {
                        output.append("File Name: ")
                              .append(file.getFileName())
                              .append("\n");
```

Main.java

```
public class Main {
    public static void main(String[] args) {
        try {
            Files.walkFileTree(folder, new SimpleFileVisitor<Path>() {
                public FileVisitResult visitFile(
                    } catch (Exception e) {
                        output.append(
                            "Could not read file information\n"
                        );
                    }

                    return FileVisitResult.CONTINUE;
                }
            });

            Files.writeString(report, output.toString());

            System.out.println(
                "File report created successfully."
            );

            System.out.println(
                "Report saved to: " + report.toAbsolutePath()
            );

        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Terminal

Output :

```
===== FILE REPORT =====
FILE REPORT
===========

File Name: Main.class
Size: 2115 bytes
Modified: 2026-10-28T19:35:50.863219062Z

File Name: Main$1.class
Size: 1570 bytes
Modified: 2026-10-28T19:35:50.797223127Z

File Name: Main.java
Size: 1690 bytes
Modified: 2026-10-28T19:35:49Z


File report created successfully.
Report saved to: /box/file_report.txt
```

Learning Outcomes – Problem 3

After completing this program, you will be able to:

1. Understand **Java NIO file handling**.
2. Use `Files.walkFileTree()` to **traverse folders and subfolders**.
3. Retrieve file **name, size, and last modified time**.
4. Use `BasicFileAttributes` to get file information.
5. Generate a **file report automatically**.
6. Save the generated report into a **text file**.